

PRÁCTICA 4

Mario VILAR
20392724

David Díez
20392643

29 de noviembre de 2023



UNIVERSITAT DE
BARCELONA

ÍNDICE

1	Introducción	2
2	Semana 9	2
	<i>Ejercicio 1</i>	2
	<i>Ejercicio 2</i>	2
	<i>Ejercicio 3</i>	3
	<i>Ejercicio 4</i>	3
	<i>Ejercicio 5</i>	3
3	Semanas 10 y 11	3
	<i>Ejercicio 6</i>	3
	<i>Ejercicio 7</i>	4
	<i>Ejercicio 8</i>	4
	<i>Observación 9</i>	6
4	Conclusiones	6
	Referencias	6

INTRODUCCIÓN

La programación concurrente y paralela se ha convertido en un pilar fundamental en el panorama de la informática moderna. Esta técnica revolucionaria permite la ejecución simultánea de múltiples tareas y procesos en entornos multiprocesador y multihilo. Optimiza el rendimiento y la eficiencia de las aplicaciones al distribuir la carga de trabajo entre diversos núcleos de CPU o incluso a través de múltiples computadoras. El objetivo de esta práctica es sumergirse en el análisis de vuelos proporcionado inicialmente y transformarlo en un proceso concurrente, haciendo uso de funciones especializadas para la creación y el control de hilos durante las etapas posteriores del curso. Debemos familiarizarnos con las herramientas y técnicas necesarias para llevar a cabo dichos procesos concurrentes.

SEMANA 9

Ejercicio 1. ¿Qué información muestra? Indica la línea de código que muestre el top 15 de los aeropuertos con más destinos de ese año ordenados de mayor a menor.

Demostración. Al ejecutar la línea `./analisis airports.csv 2007.csv` obtenemos una lista de cada aeropuerto y la cantidad de destinos que tiene. El formato es el siguiente:

```
Origin: ABQ -- Number of different destinations: 39
```

Si ahora queremos mostrar los 15 aeropuertos con más destinos podemos utilizar la línea:

```
./analisis airports.csv 2007.csv | sort -t ":" -k 3nr | head -15
```

y el resultado obtenido es

```
Origin: ATL -- Number of different destinations: 180
Origin: ORD -- Number of different destinations: 153
Origin: DFW -- Number of different destinations: 132
Origin: MSP -- Number of different destinations: 129
Origin: DEN -- Number of different destinations: 121
Origin: SLC -- Number of different destinations: 120
Origin: IAH -- Number of different destinations: 117
Origin: CVG -- Number of different destinations: 115
Origin: DTW -- Number of different destinations: 115
Origin: EWR -- Number of different destinations: 92
Origin: LAS -- Number of different destinations: 92
Origin: LAX -- Number of different destinations: 91
Origin: PHX -- Number of different destinations: 88
Origin: MCO -- Number of different destinations: 82
Origin: MEM -- Number of different destinations: 78
```

Y con ello ya hemos acabado. ✓

Ejercicio 2. ¿Cuánto tiempo tarda en ejecutarse cada uno? ¿Con qué comando podrías averiguarlo (ignorando el tiempo que ya devuelve el programa)?

Demostración. En efecto, si utilizamos el archivo del 1990 obtenemos:

```
real 0m31,895s
user 0m27,723s
sys 0m2,49s
```

mientras que para ejecutar el del 2007 tardamos:

```
real 0m44,071s
user 0m38,563s
sys 0m3,084s
```

Para obtener estos tiempos hemos utilizado el comando `time ./analisis airports.csv yyyy.csv` donde `yyyy` hace referencia al año del fichero. ✓

Ejercicio 3. ¿Qué comandos puedes utilizar para calcular el número de vuelos que contiene cada uno de ellos? ¿Cuántos vuelos contienen cada uno de ellos?

Demostración. Para calcular el número de vuelos totales utilizamos el comando `cat yyyy.csv | wc -l` donde `yyyy` hace referencia al año del fichero. Para el `1990.csv` obtenemos 5270894 vuelos y para el `2007.csv` un total de 7453216. ✓

Ejercicio 4. ¿Cuáles son las funciones que más tardan en ejecutarse? ¿Cuál es el porcentaje de tiempo que ocupa la ejecución de cada una de las funciones?

Demostración. Antes de todo, se recomienda lo siguiente antes de ejecutar el programa (cf. [Val23]): *Compile your program with -g to include debugging information so that Memcheck's error messages include exact line numbers. Using -O0 is also a good idea, if you can tolerate the slowdown.* Ordenamos las funciones según *Self* o *Incl.* para ver cuáles consumen más tiempo en términos de tiempo propio (dentro de la función) o incluyendo tiempo de funciones llamadas.

- La columna «Self» muestra el tiempo que una función específica pasa en su propio código, excluyendo el tiempo consumido por las funciones que llama.
- La columna «Incl.» (incluido) muestra el tiempo total que una función pasa ejecutando su propio código, más el tiempo consumido por todas las funciones que se llaman directa o indirectamente desde esa función.

Por ello, funciones como `0x000...` (100 %) y `main` (99,9 %) son aquellas con un valor *Incl.* más alto, porque todo el programa corre desde ellas. Cabe destacar `read_airports_data` (98,48 %) y `get_index_airport` (84,58 %). Y por ello también, merece la pena fijarse en las *Self*. Las que más tiempo consumen son: el comparador de strings `_strcmp_avx2` (48,06 %), `get_index_airport` (32,48 %) y `extract_fields_airport` (11,58 %). ✓

Ejercicio 5. ¿Qué errores se observaron a través de la salida del Valgrind? ¿Cómo los has corregido?

Demostración. Después de analizar con detenimiento el archivo `valgrind.out` originado del comando hemos visto un par de errores importantes.

```
valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes --verbose \
--log-file=valgrind-out.txt ./analisis airports.csv file_test.csv
```

1. Valores no inicializados: Se observa que hay código que depende de variables que no se han inicializado aún en `print_num_flights_summary` lo que puede conducir a un mal comportamiento del programa. Sustituimos `malloc` por `calloc` en `malloc_matrix`.
2. Memory leaks: Valgrind ha detectado dos bloques de memoria (ambos de 2424 bytes) que se crean dinámicamente a través del `malloc` pero los cuales no han sido liberados con `free` antes de acabar el programa. Estas podrían ser potenciales fugas de memoria. Hemos hecho `free(matrix)` en la función de liberar la matriz. ✓

3 SEMANAS 10 Y 11

Agradecer los PDFs del doctor Garrido, que nos han sido de gran ayuda para implementar el código de estas dos semanas.

Ejercicio 6. ¿Qué variables debería pasar por argumento el hilo principal a los secundarios?

Demostración. Después de leer la cabecera del archivo `read_airports_data` y antes de comenzar a procesar los datos debemos crear mediante `pthread_create` dos hilos secundarios. Para procesar los datos del archivo, necesitamos un conjunto de variables. Hemos implementado un `struct`, al que hemos llamado `params`, que agrupa este entramado de parámetros:

1. `FILE *fp`: simplemente un *file pointer*, una variable que se utiliza para referenciar un fichero abierto; una estructura que almacena los datos del fichero. Es importante para acceder a ellos y cargarlos a la matriz que usaremos durante la ejecución.

1. `lines`: El total de líneas que debemos leer con el hilo que corresponda, las cuales acaban guardadas en una variable propia del hilo. En el enunciado se habla de N .
3. `num_flights`: esta matriz de enteros está compartida entre los dos hilos, que la irán actualizando concurrentemente (siempre que evitemos, eso sí, la interferencia que se pueda producir). Se encarga de monitorizar el número de vuelos entre cada par de aeropuertos.
4. `airports`: esta matriz de `char` almacena, evidentemente, el conjunto de aeropuertos. Notar que *nunca* vamos a modificar esta matriz; es decir, es sólo de consulta. Nos permitirá rellenar `num_flights` acordemente. ✓

Ejercicio 7. ¿Qué partes del código (secciones críticas) has protegido para evitar interferencias? Justifica tu respuesta.

Demostración. Todas las secciones críticas de nuestro programa se encuentran en la función de hilos, `thread_function`.

1. Leemos las líneas del archivo hasta que alcanzamos el límite de líneas especificado;
2. Para cada línea, extraemos el origen y destino del vuelo usando `extract_fields_airport`;
3. Obtenemos los índices respectivos en la matriz de aeropuertos con `get_index_airport`;
4. Incrementamos el contador de vuelos entre esos dos aeropuertos si los índices son válidos.

Ello lo tenemos que hacer con sumo cuidado, en particular las etapas 1. y 4., para evitar la ya mencionada *interferencia*. Esto es porque ambos hilos acceden y actualizan la matriz `num_flights` y sin `mutex` se podrían producir *race conditions*, de manera que podríamos acabar con información incorrecta o incompleta.

El uso de `pthread_mutex_lock` y `pthread_mutex_unlock` asegura que solo un hilo acceda y modifique la sección crítica (la actualización de `num_flights`) a la vez, evitando problemas de concurrencia y manteniendo la consistencia de los datos compartidos entre hilos, [Fin23]. ✓

Ejercicio 8. Muestra una tabla mostrando los tiempos de ejecución para diferente número de hilos ($H \in \{1, 2, 3, 4, 6, 8\}$), así como diferentes líneas por bloque ($N \in \{1, 10, 100, 1000, 10000, 100000\}$). Comenta los resultados.

Demostración. Para hacer valer nuestra involucración en la asignatura, nos hemos entretenido en hacer un programa en Python que nos automatice el proceso, de forma que podremos disponer de los datos necesarios en nuestro formato preferido.

```
python3 iterables.py <ejecutable> <airports.csv> <flights.csv> <número de hilos> <número de líneas/hilo>
```

Hemos adaptado el fichero `analisis.c` en concordancia, de manera que podremos compilarlo y ejecutarlo con los mismos comandos que hasta ahora así lo queremos. Empezamos por el principio, la estructura es la siguiente:

1. El comando acepta los argumentos que sea, pero como máximo lee cinco parámetros, mínimo tres: el primero es el ejecutable que hemos generado con el `Makefile`. El segundo y tercero son los mismos que los usados en la semana 9 (cf. sección 2, ejercicio 1), `airports.csv` y el `flights.csv` de nuestra elección.
2. El cuarto y quinto parámetro son opcionales; si se reciben ambos se ejecuta el programa de gestión de vuelos con un número de hilos y líneas del bloque por cada hilo fijado por la entrada que se dé, en ese orden.
3. Si no se reciben, se entiende que el usuario quiere calcular la matriz que se nos pide en 8, es decir, para cada par (H, N) ($H \in \{1, 2, 3, 4, 6, 8\}$ y $N \in \{1, 10, 100, 1000, 10000, 100000\}$) encontrar el tiempo de ejecución y no solo eso: fijado un número de iteraciones concreto, calcularemos la media y la desviación estándar¹. Devolveremos estos dos valores en formato tupla, ambos redondeados a seis decimales; es decir, (i, j) será un par (M, S) , M de mean y S de *standard deviation*, para los valores $(H = i, N = j)$.

Ahora debemos ver la matriz resultante para cada fichero de vuelos, es decir, `1990.csv` por un lado, `2007.csv` por el otro. Al tener 2007 más vuelos los tiempos de ejecución serán más altos. Como trabajamos en $\text{\LaTeX}$ hemos hecho que el formato de la matriz que devuelva por pantalla ya sea el correcto.

¹ No pasaremos este número de iteraciones como argumento ya que hemos preferido no abusar de ellos; llevamos cinco hasta este momento, dos de ellos también enteros. Sería demasiado fácil confundirse y perderíamos legibilidad.

	I	IO	IOO	IOOO	IOOOO	IOOOOO
1	(0,023635, 0,000227)	(0,033097, 0,001125)	(0,03667, 0,000319)	(0,042325, 0,000744)	(1,332803, 0,069372)	(76,199747, 4,179249)
2	(0,023425, 0,000129)	(0,023185, 0,000405)	(0,026546, 0,000647)	(0,045394, 0,003194)	(1,448549, 0,050749)	(87,277232, 2,313392)
3	(0,023448, 0,000341)	(0,021245, 0,000314)	(0,024509, 0,000315)	(0,049627, 0,002977)	(1,649476, 0,047684)	(101,19714, 5,876378)
4	(0,023519, 0,000268)	(0,019885, 0,000797)	(0,022884, 0,000354)	(0,05005, 0,000989)	(1,736038, 0,018475)	(114,974222, 3,323507)
6	(0,023064, $5,6 \cdot 10^{-5}$)	(0,018053, 0,000669)	(0,020596, 0,000278)	(0,048646, 0,001794)	(1,887085, 0,038522)	(125,962393, 3,187598)
8	(0,023739, 0,000349)	(0,018213, 0,000972)	(0,02058, 0,000238)	(0,051417, 0,001311)	(1,94251, 0,011983)	(140,518537, 4,876684)

(3.1)

Tabla 1: Salida para file-test.csv.

	I	IO	IOO	IOOO	IOOOO	IOOOOO
1	(6,380561, 0,073668)	(2,540587, 0,043123)	(2,694192, 0,007318)	(2,922032, 0,025284)	(3,152793, 0,054374)	(86,267461, 5,620583)
2	(6,307859, 0,009383)	(2,102434, 0,014386)	(2,1591, 0,016732)	(2,340948, 0,020167)	(2,548709, 0,017068)	(97,34699, 2,172784)
3	(6,331848, 0,035856)	(1,985149, 0,030475)	(1,966291, 0,003206)	(2,156125, 0,014121)	(2,352533, 0,027245)	(103,008888, 2,465876)
4	(6,298748, 0,018872)	(1,920855, 0,008861)	(1,889915, 0,013439)	(2,066267, 0,012716)	(2,250929, 0,018084)	(109,825438, 2,192024)
6	(6,291279, 0,013962)	(1,891346, 0,0158)	(1,814471, 0,014834)	(1,983262, 0,014364)	(2,196924, 0,035842)	(119,696781, 2,205822)
8	(6,283706, 0,004909)	(1,886535, 0,028142)	(1,790593, 0,007095)	(1,956589, 0,015672)	(2,16709, 0,022846)	(121,558989, 0,499333)

(3.2)

Tabla 2: Salida para 1990.csv.

	I	IO	IOO	IOOO	IOOOO	IOOOOO
1	(8,908423, 0,033086)	(3,539514, 0,053625)	(3,735625, 0,048584)	(4,115109, 0,026083)	(4,343845, 0,061124)	(103,666011, 6,958494)
2	(8,880894, 0,00737)	(2,986942, 0,035478)	(3,215593, 0,025887)	(3,4563, 0,01131)	(3,579611, 0,098932)	(110,649284, 2,094138)
3	(8,870315, 0,013346)	(2,738742, 0,024261)	(2,838251, 0,013873)	(3,085228, 0,018182)	(3,299106, 0,040294)	(114,32323, 0,962427)
4	(8,888237, 0,025561)	(2,66383, 0,013109)	(2,663985, 0,019208)	(2,939064, 0,013135)	(3,166547, 0,045362)	(122,194487, 5,307063)
6	(8,878094, 0,012608)	(2,630727, 0,01212)	(2,57537, 0,009222)	(2,822682, 0,020991)	(3,065502, 0,041703)	(131,852038, 3,482521)
8	(8,874307, 0,014004)	(2,632104, 0,006115)	(2,525085, 0,020535)	(2,803669, 0,011621)	(2,993328, 0,010363)	(138,368623, 2,359289)

(3.3)

Tabla 3: Salida para 2007.csv.

Adjuntaremos el fichero Python en el código base de la práctica. Tiene que estar todo (.csv, .c, .py) en el mismo directorio.

Respecto a los resultados obtenidos, vemos claramente como a medida que aumenta el número de vuelos del fichero el tiempo de ejecución también se ve afectado. Las diferencias más notables se encuentran entre $N = 1$ y el resto (cf. comentarios de resultado, 1.). Vemos cómo tanto el tiempo como la desviación estándar aumentan dramáticamente. De hecho, hemos hecho 5 iteraciones para todos los casos. **Ahora, algunos comentarios sobre los resultados obtenidos.**

1. Para $N = 1$ tenemos lo mismo que si no utilizáramos programación concurrente, ya que utilizamos un solo hilo, de manera que para la columna $N = 1$ se van a obtener tiempos generalmente más altos que en el resto de casos, $N \in \{10, 1000, 10000, 100000\}$. Esto se hace notar en los ficheros *grandes*, es decir, 1990.csv y 2007.csv.
2. También para 1990 y 2007 se puede apreciar que en una misma columna, es decir, para un mismo número de líneas por hilo, el tiempo de ejecución disminuye a medida que aumentamos N , eso es, los hilos utilizados. Podríamos concluir que **el uso de hilos hace el programa más eficiente**. Parece que no pasa lo mismo para *file_test.csv*.
3. Nos ha costado mucha *paciencia* encontrar datos estadísticamente relevantes para $N = 100000$. Esto provoca que ejecutar el procedimiento que explicábamos en 3. (pág. 4) se haga complicado. Creemos que su alto tiempo de ejecución es debido a que 100000 es el número de líneas del fichero, de manera que múltiples hilos leyendo tantas líneas no sólo no es eficiente, sino que es totalmente innecesario. Dejamos los resultados indicados para todos los ficheros, tablas 1, 2 y 3. Como comentario, para *file_test*, un fichero ínfimo, ha tardado *un minuto* ($(H, N) = (2, 1000)$) para una iteración; para 5 iteraciones, $N = 100000$ y $H \in \{1, 2, 3, 4, 6, 8\}$, alrededor de una hora.
Para $N = 100000$ es relevante notar que, como más hilos usemos aumenta el tiempo de ejecución. Esto es totalmente contradictorio a lo que hemos visto para el resto de casos y hemos explicado más arriba, en 1990 y 2007, $N \in \{10, 1000, 10000\}$. Además, los valores de desviación estándar también aumentan dramáticamente. ✓

Observación 9. Por alguna razón, parece que este script no funciona para valores $N = 100000$; en particular, en la máquina virtual. Mario lo ha ejecutado correctamente en su MacBook. Sea como sea, hemos puesto los resultados que hemos podido en las mismas tablas 1, 2 y 3.

4

CONCLUSIONES

La experiencia adquirida durante esta práctica ha permitido comprender la importancia y el impacto de la programación concurrente y paralela en el desarrollo de aplicaciones modernas. Hemos visto, en particular, cuánto puede optimizar el uso de varios hilos la lectura de un fichero. De esta forma, hemos optimizado temporalmente un programa secuencial. Explorar y aplicar funciones especializadas para la creación y el control de hilos ha proporcionado una visión clara sobre cómo aprovechar al máximo las capacidades de los sistemas multiprocesador y multihilo para alcanzar resultados más eficientes. Hablamos, en concreto, de `pthread_create`, `pthread_join` y el mutex.

La repartición de tareas ha sido lo más igualitaria posible. Hicimos la semana 9 juntos y el código de las semanas 10 y 11. Mario ha confeccionado el script de Python.

REFERENCIAS

- [Fre21] FREE SOFTWARE FOUNDATION, INC. *GNU Make Manual*. Accessed on 29 de noviembre de 2023. Free Software Foundation, Inc. 2021. URL: <https://www.gnu.org/software/make/manual/make.html>.
- [AA23] Remzi H. ARPACI-DUSSEAU y Andrea C. ARPACI-DUSSEAU. *Operating Systems: Three Easy Pieces*. 1.10. Arpaci-Dusseau Books, nov. de 2023.
- [Fin23] Ian FINLAYSON. *Mutexes*. <https://ianfinlayson.net/class/cpsc425/notes/05-mutexes>. 2023.
- [Val23] VALGRIND. *Valgrind: Quick Start Guide*. <https://valgrind.org/docs/manual/quick-start.html>. Página web, acceso: 29 de noviembre de 2023. 2023.